The ECDSA UAIR$^+$

As implemented in the Zinc+ reference codebase

Abstract

We describe the Universal AIR with lookups (UAIR$^+$) instance that arithmetizes ECDSA signature verification on the `secp256k1` elliptic curve, as deployed in the Zinc+ reference implementation. We follow the conventions and notation of the main Zinc+ paper, and focus on the design choices made by the implementation: namely, *(i)* native arithmetization in the $\mathbb{F}_p$ branch (no $\mathbb{F}_2[X]$ or bit-polynomial columns); *(ii)* Shamir's trick to merge the two scalar multiplications into a single 256-row double-and-add loop; *(iii)* Jacobian coordinates throughout the loop, with the field inversion required for the affine readout deferred off-protocol; *(iv)* per-row addend coordinates supplied by the verifier (rather than committed and bound in-circuit), reducing the witness to 8 field-element columns and the constraint count to 11.

Contents

1 ECDSA on `secp256k1` and the target relation **1**

2 Arithmetizing the ECDSA building blocks **3**
- 2.1 Shamir's trick . . . 3
- 2.2 Jacobian doubling . . . 4
- 2.3 Mixed Jacobian-affine addition . . . 4
- 2.4 Conditional addition via output-selection . . . 4
- 2.5 Initial step and final state . . . 5

3 Full UAIR$^+$ arithmetization **5**
- 3.1 Trace shape . . . 5
- 3.2 Column layout . . . 5
- 3.3 Constraints . . . 6
- 3.4 Algebraic-degree summary . . . 7

4 Arithmetization summary **8**
- 4.1 Why 8 committed witness columns suffice . . . 8
- 4.2 What the public input absorbs . . . 8
- 4.3 Composition with downstream UAIRs . . . 9

1 ECDSA on `secp256k1` and the target relation

We briefly recall the ECDSA signature verification procedure on the `secp256k1` curve, then state the relation our UAIR$^+$ arithmetizes.

The `secp256k1` curve. Let p be the 256-bit `secp256k1` base prime and let $E/\mathbb{F}_p$ denote the short Weierstrass curve $y^2 = x^3 + 7$. Let $G \in E(\mathbb{F}_p)$ be the standard base point and let n be its prime order (also 256-bit, with $p < 2n$). The cyclic subgroup $\langle G \rangle$ is the order-n subgroup in which all signatures live.

ECDSA verification. A signature on a message $e \in \mathbb{F}_n$ under public key $Q \in \langle G \rangle$ is a pair $(r, s) \in \mathbb{F}_n \times \mathbb{F}_n^*$. The verifier computes

$$u_1 := e \cdot s^{-1} \bmod n, \qquad u_2 := r \cdot s^{-1} \bmod n,$$

forms the point

$$R := u_1 G \;+\; u_2 Q \;\in\; E(\mathbb{F}_p),$$

and accepts iff $R \neq \mathcal{O}$ and the affine x-coordinate R_x satisfies $\mathrm{int}(R_x) \equiv r \pmod{n}$.

Jacobian coordinates. Throughout the arithmetization we work in Jacobian coordinates: a triple $(P_X : P_Y : P_Z)$ with $P_Z \neq 0$ represents the affine point $(P_X/P_Z^2,\, P_Y/P_Z^3)$, and any triple with $P_Z = 0$ represents $\mathcal{O}$. Jacobian doubling and mixed addition reduce to a small number of $\mathbb{F}_p$-multiplications and squarings without field inversions, deferring the single inversion to a final affine readout.

Target relation. We arithmetize the multi-scalar elliptic-curve multiplication underlying ECDSA verification. The remaining checks ($u_1 \cdot s = e$ and $u_2 \cdot s = r$ in $\mathbb{F}_n$, and the final $R_x \equiv r \pmod{n}$ test) are negligible in cost and are performed by the verifier off-protocol against the witness final-row state. Concretely, the relation is

$$\mathrm{REL}_{\text{ECDSA-MSM}} := \left\{ (\beth, \curvearrowleft; \precapprox) \;\middle|\; \begin{array}{l} \beth = (G), \\ \curvearrowleft = (Q,\ G+Q,\ b_1,\ b_2,\ R_{\text{init}}), \\ \precapprox = (R), \\ Q \in \langle G \rangle,\ b_1, b_2 \in \{0,1\}^{256}, \\ R_{\text{init}} \in E(\mathbb{F}_p) \setminus \{\mathcal{O}\}, \\ R \;=\; R_{\text{init}}^{(2^{256})} \;+\; \displaystyle\sum_{t=1}^{256} 2^{256-t}(b_1[t]G + b_2[t]Q). \end{array} \right\}. \tag{1}$$

Here b_1, b_2 are the bit-decompositions of u_1, u_2 supplied as public input by the verifier (after the verifier computes u_1, u_2 from (e, r, s) off-protocol), and the R_{init} is a non-identity Jacobian seed point that side-steps the identity-aware initial step problem discussed below.

Remark 1.1 (u_1, u_2 and R_{init} supplied by the verifier). We place u_1, u_2 — and hence their bit-decompositions b_1, b_2 — in the public input rather than the witness. The verifier is responsible for checking $u_1 \cdot s \equiv e \pmod{n}$, $u_2 \cdot s \equiv r \pmod{n}$, and for verifying the bit decompositions $\sum_{t=1}^{256} b_i[t] \cdot 2^{256-t} = u_i$ in $\mathbb{F}_n$, all off-protocol. This shifts the two $\mathbb{F}_n$ multiplications and the two bit-decomposition checks out of the arithmetization, removing the need for an $\mathbb{F}_n$-trace and any $\{0,1\}^{<32}[X]$ columns. The cost saving is significant; the privacy cost is that u_1, u_2 are not hidden from the proof. In scenarios where any of e, r, s must remain private, u_1, u_2 would have to be moved back into the witness and the corresponding well-formedness constraints reinstated. The same applies to other components currently in the public input (notably Q and $G + Q$).

Remark 1.2 (Why a non-identity R_{init}). The honest scalar-multiplication algorithm starts from the Jacobian identity $\mathcal{O} = (1 : 1 : 0)$. The mixed-addition formulas used in our double-and-add loop (Section 2.3) become singular when their left summand is $\mathcal{O}$ (specifically, $Z = 0$ collapses both $H = X_{\text{add}} Z^2 - X$ and $R_a = Y_{\text{add}} Z^3 - Y$ to zero), so the loop cannot be initialized at $\mathcal{O}$. The implementation side-steps this by having the verifier supply a non-identity Jacobian seed R_{init} at row 0 and offsetting the final result by the appropriately doubled seed, expressed in $\mathrm{REL}_{\text{ECDSA-MSM}}$ via the $R_{\text{init}}^{(2^{256})}$ term. Replacing the seed by unified addition formulas (which handle the identity input cleanly) is left as a follow-up.

Off-protocol checks. The remaining ECDSA verification checks are performed by the verifier off-protocol against the witness final-row Jacobian state $R = (R_X : R_Y : R_Z) \in \mathbb{F}_p^3$:

- $R \neq \mathcal{O}$, equivalently $R_Z \neq 0$;
- $R_x := R_X \cdot R_Z^{-2} \bmod p$ (single field inversion);
- $\mathrm{int}(R_x) \equiv r \pmod{n}$.

None of these are enforced in-circuit. The implementation retains the $\mathsf{S_FINAL}$ selector in the column layout so that downstream UAIRs composing with this one (e.g. a gluing UAIR that does enforce the affine readout in-circuit, or one that connects the digest of a SHA-256 UAIR to e) can still gate boundary constraints on the same row.

The remainder of this document constructs a UAIR$^+$ instance $(\beth_{\mathrm{uair}}, \curvearrowleft_{\mathrm{uair}}; \precsim_{\mathrm{uair}})$ that lies in $\mathrm{REL}_{\mathrm{UAIR}^+}$ if and only if the input witness lies in $\mathrm{REL}_{\mathrm{ECDSA\text{-}MSM}}$.

2 Arithmetizing the ECDSA building blocks

We collect the arithmetic identities that translate each step of the double-and-add loop into a constraint over $\mathbb{F}_p$. Throughout this section, all arithmetic is in $\mathbb{F}_p$ unless noted otherwise; we suppress reductions $\bmod p$.

2.1 Shamir's trick

The standard binary double-and-add algorithm computes a single scalar multiplication uA (for a k-bit scalar u and a point A) by initializing an accumulator $P \leftarrow \mathcal{O}$ and, for $j = k-1, \ldots, 0$, performing

$$P \leftarrow 2P, \qquad P \leftarrow P + A \quad \text{whenever bit } j \text{ of } u \text{ is } 1.$$

Naively, computing the two-term sum $u_1 G + u_2 Q$ would run two such loops for a total of $2k$ doublings and up to $2k$ additions. Shamir's trick merges them into a single loop. Precompute the four points

$$T_{00} = \mathcal{O}, \quad T_{10} = G, \quad T_{01} = Q, \quad T_{11} = G + Q,$$

initialize $P \leftarrow \mathcal{O}$, and run, for $j = k-1, \ldots, 0$,

$$P \leftarrow 2P, \qquad P \leftarrow P + T_{b_1[j]\, b_2[j]}.$$

After $k = 256$ steps, $P = u_1 G + u_2 Q$. The total cost is k doublings and at most k mixed additions.

In the UAIR, the per-row affine addend $T_{b_1[t]\, b_2[t]} \in \{\mathcal{O}, G, Q, G+Q\}$ is supplied by the verifier in two public columns $\mathsf{PA_X_ADDEND}$ and $\mathsf{PA_Y_ADDEND}$, with a separate boolean public selector $\mathsf{S_ADD}$ flagging whether the addend is non-identity (i.e. $\mathsf{S_ADD}[t] = 1 \iff (b_1[t], b_2[t]) \neq (0,0)$). When $\mathsf{S_ADD}[t] = 0$ the addition step at row t collapses to $P \leftarrow 2P$.

Remark 2.1 (Verifier-supplied addend). Q and $G+Q$ are public (Theorem 1.1), so the four-point precompute $\{\mathcal{O}, G, Q, G+Q\}$ is public, and the per-row addend choice is determined by the public bits $(b_1[t], b_2[t])$. The implementation has the verifier write the chosen affine addend directly into $(\mathsf{PA_X_ADDEND}, \mathsf{PA_Y_ADDEND})$ for each row, rather than committing four selector polynomials and adding in-circuit constraints to compute the addend from the bits. The saving is large: the alternative would require four multiplicative selectors per row and would push the addition-formula degrees up further. The cost is that the verifier must do this 256-row table lookup itself before committing to the public input.

2.2 Jacobian doubling

Given an input Jacobian point $(P_X : P_Y : P_Z)$, the Jacobian doubling $2P = (P_{\mathrm{pa},X} : P_{\mathrm{pa},Y} : P_{\mathrm{pa},Z})$ is computed in $\mathbb{F}_p$ by

$$P_{\mathrm{pa},Z} = 2P_Y P_Z, \tag{2}$$
$$P_{\mathrm{pa},X} = 9P_X^4 - 8P_X P_Y^2, \tag{3}$$
$$P_{\mathrm{pa},Y} = 12P_X^3 P_Y^2 - 3P_X^2 P_{\mathrm{pa},X} - 8P_Y^4. \tag{4}$$

These are the standard short-Weierstrass $a = 0$ doubling formulas. The intermediate $S = P_Y^2$ is computed inline (the implementation inlines it rather than committing it as a separate column).

2.3 Mixed Jacobian-affine addition

Given $(P_{\mathrm{pa},X} : P_{\mathrm{pa},Y} : P_{\mathrm{pa},Z})$ (the doubled point) and an affine addend (T_X, T_Y) from the precomputed $\{G, Q, G+Q\}$, the mixed addition $P_{\mathrm{pa}} + T = (P_{\mathrm{add},X} : P_{\mathrm{add},Y} : P_{\mathrm{add},Z})$ proceeds via the scratch quantities

$$H := T_X \cdot P_{\mathrm{pa},Z}^2 \; - \; P_{\mathrm{pa},X}, \tag{5}$$
$$R_a := T_Y \cdot P_{\mathrm{pa},Z}^3 \; - \; P_{\mathrm{pa},Y}, \tag{6}$$

followed by the addition outputs

$$P_{\mathrm{add},Z} = P_{\mathrm{pa},Z} \cdot H, \tag{7}$$
$$P_{\mathrm{add},X} = R_a^2 - H^3 - 2P_{\mathrm{pa},X} H^2, \tag{8}$$
$$P_{\mathrm{add},Y} = R_a(P_{\mathrm{pa},X} H^2 - P_{\mathrm{add},X}) - P_{\mathrm{pa},Y} H^3. \tag{9}$$

The implementation commits only H (named W_C) and R_a (named W_D), inlining $P_{\mathrm{pa},Z}^2$, $P_{\mathrm{pa},Z}^3$, H^2, H^3, and $P_{\mathrm{pa},X} H^2$ into the surrounding constraint expressions. This trades higher per-constraint algebraic degree for fewer committed columns; with the inlined form the maximum constraint degree is 6 (attained in (9) after selector gating, see Section 3.3).

2.4 Conditional addition via output-selection

Rather than gating the addition step with a multiplicative selector that would inflate the constraint degree, the implementation folds the conditional add into the row-chaining itself. Let $\mathsf{S_ADD}[t] \in \{0, 1\}$ be the per-row addition selector (Theorem 2.1). The next-row Jacobian state is then defined as

$$P_{\mathrm{next},X} = P_{\mathrm{pa},X} + \mathsf{S_ADD}(P_{\mathrm{add},X} - P_{\mathrm{pa},X}), \tag{10}$$
$$P_{\mathrm{next},Y} = P_{\mathrm{pa},Y} + \mathsf{S_ADD}(P_{\mathrm{add},Y} - P_{\mathrm{pa},Y}), \tag{11}$$
$$P_{\mathrm{next},Z} = P_{\mathrm{pa},Z} + \mathsf{S_ADD}(P_{\mathrm{add},Z} - P_{\mathrm{pa},Z}), \tag{12}$$

so that $\mathsf{S_ADD} = 1$ selects P_{add} and $\mathsf{S_ADD} = 0$ selects P_{pa} (the pure doubling result).

Because $\mathsf{S_ADD}$ is a public boolean, $\mathsf{S_ADD}(1 - \mathsf{S_ADD}) = 0$ need not be enforced in-circuit; the verifier supplies the column with 0/1 values directly.

Remark 2.2 (Inlined addition collapses to a single output-selection constraint). Substituting $P_{\mathrm{add},X/Y/Z}$ from (7)–(9) into (10)–(12) and inlining all of $P_{\mathrm{pa},Z}^2, H^2, H^3, P_{\mathrm{pa},X} H^2$ yields three constraints whose maximum algebraic degree (in committed columns) is 6. The deg-6 monomial in the Y output-selection constraint is exactly $\mathsf{S_ACTIVE} \cdot \mathsf{S_ADD} \cdot D \cdot X_{\mathrm{pa}} \cdot C^2$, matching the spec's $s_{\mathrm{reg}} \cdot R_a \cdot X_{\mathrm{mid}} \cdot H^2$ term.

2.5 Initial step and final state

Init boundary. At row 0 the Jacobian state is pinned to the verifier-supplied non-identity seed $R_{\text{init}} = (R_{\text{init},X} : R_{\text{init},Y} : R_{\text{init},Z})$ (Theorem 1.2):

$$(P_X[0], P_Y[0], P_Z[0]) \;=\; (\mathsf{PA_R_INIT_X}, \mathsf{PA_R_INIT_Y}, \mathsf{PA_R_INIT_Z}).$$

This is enforced by three zero-ideal constraints gated by $\mathsf{S_INIT}$ (see Equation (I1)–Equation (I3)).

Final state and off-protocol affine readout. At the final row $t = \text{FINAL_ROW} := \text{NUM_SHAMIR_ROUNDS} = 256$, the witness Jacobian state $(P_X : P_Y : P_Z)[256]$ holds the result $R = u_1 G + u_2 Q$ (modulo the R_{init} offset of Theorem 1.2). The affine x-coordinate $R_x = P_X[256] \cdot P_Z[256]^{-2} \bmod p$ is computed *off-protocol* by the verifier, who opens the three witness cells at row 256, performs a single field inversion, and checks $R_x \neq \mathcal{O}$ (i.e. $P_Z[256] \neq 0$) and $\text{int}(R_x) \equiv r \pmod{n}$. Consequently, the UAIR has *no Z_{inv} column, no in-circuit affine conversion, and no signature check $R_x \equiv r$.* The selector column $\mathsf{S_FINAL}$ is retained in the layout for downstream UAIRs that compose with this one and want to gate additional final-row constraints.

3 Full UAIR$^+$ arithmetization

We assemble the building blocks of Section 2 into a single UAIR$^+$ instance. As in the SHA UAIR, we use the *shift-access* extension of the main paper (Remark 2.2 of the Zinc+ paper); we do *not* need the affine-combination lookup extension because the ECDSA UAIR has no lookup constraints.

3.1 Trace shape

The trace runs NUM_SHAMIR_ROUNDS := 256 Shamir double-and-add iterations followed by one final row that holds the output Jacobian state. Concretely, with FINAL_ROW := NUM_SHAMIR_ROUNDS = 256, the active row range is $t \in [0, 256]$ (257 active rows total) and the trace length is the smallest power of 2 with $2^\nu \geq 257$, namely $n := 2^9 = 512$.

Per row t, the trace processes one Shamir step $P[t+1] = 2P[t] + T[t]$ (with $T[t] \in \{\mathcal{O}, G, Q, G+Q\}$ chosen by the bit pair $(b_1[t], b_2[t])$); the final row 256 holds the result R for the off-protocol affine readout.

3.2 Column layout

Following Definition 8.1 of the main paper, our index has $\mathbf{q} = (p)$ where p is the `secp256k1` base prime, so we have a single $\mathbb{F}_p$ branch. *All* witness and constraint columns live in this branch; there is no $\mathbb{Q}[X]$ branch and no binary-polynomial section. Each trace cell is an $\mathbb{F}_p$ field element (i.e. $d_p = 1$, treating $\mathbb{F}_p[X]$ at degree 0).

Tables 1 and 2 list the 9 public and 8 witness columns, respectively. We write $P[t]$ for the entry at row t of column P and $P^{\downarrow}[t] := P[t+1]$ for shifted access.

Remark 3.1 (No range checks). Because every column is a native $\mathbb{F}_p$ field element and the proving field equals the `secp256k1` base prime, no range-check or lookup constraint is required. This is the principal reason the ECDSA UAIR has no lookup PIOP attached and no $\{0,1\}^{<32}[X]$ columns: arithmetic over $\mathbb{F}_p$ is direct.

Remark 3.2 (Inlined intermediates). The higher-degree intermediates $S = P_Y^2$, P_Y^4, $P_{\text{pa},Z}^2$, $P_{\text{pa},Z}^3$, H^2, H^3, and $P_{\text{pa},X} H^2$ are *not* stored as committed columns; they appear inlined inside the constraint expressions. Likewise the addition outputs $P_{\text{add},X/Y/Z}$ from (7)–(9) are not committed — they appear inlined inside the three output-selection constraints (10)–(12). This trades higher

Column	Role	Looku
Public columns (9 — entire ECDSA UAIR is public-input–gated)		
S_INIT	Selector: 1 at row 0, else 0	$\{0,1\}$
S_ACTIVE	Selector: 1 for $t \in [0, 256)$, else 0	$\{0,1\}$
S_FINAL	Selector: 1 at FINAL_ROW $= 256$, else 0 (retained for composition; unused in-circuit)	$\{0,1\}$
S_ADD	Selector: 1 iff $(b_1[t], b_2[t]) \neq (0,0)$, else 0	$\{0,1\}$
PA_X_ADDEND	Affine x-coordinate of the per-row addend $T[t] \in \{G, Q, G+Q\}$ (Theorem 2.1)	$\mathbb{F}_p$
PA_Y_ADDEND	Affine y-coordinate of the per-row addend	$\mathbb{F}_p$
PA_R_INIT_X	Initial Jacobian X-coordinate of R_{init} at row 0	$\mathbb{F}_p$
PA_R_INIT_Y	Initial Jacobian Y-coordinate of R_{init} at row 0	$\mathbb{F}_p$
PA_R_INIT_Z	Initial Jacobian Z-coordinate of R_{init} at row 0	$\mathbb{F}_p$

Table 1: Public columns (9 total). All are $\mathbb{F}_p$-valued; the four selectors are boolean and verifier-derivable from the public bits b_1, b_2.

Column	Role	Lookup set $\subseteq$ Dom
Witness columns (8 — all native $\mathbb{F}_p$ field elements, no range checks)		
P_X	Chained Jacobian X-coordinate of $P[t]$ (input at row t, output via $P_X^{\downarrow}$)	$\mathbb{F}_p$
P_Y	Chained Jacobian Y-coordinate of $P[t]$	$\mathbb{F}_p$
P_Z	Chained Jacobian Z-coordinate of $P[t]$	$\mathbb{F}_p$
$P_{\text{pa},X}$	Doubled-point X-coordinate $X(2P[t])$	$\mathbb{F}_p$
$P_{\text{pa},Y}$	Doubled-point Y-coordinate $Y(2P[t])$	$\mathbb{F}_p$
$P_{\text{pa},Z}$	Doubled-point Z-coordinate $Z(2P[t])$	$\mathbb{F}_p$
H	Mixed-addition scratch $H = T_X \cdot P_{\text{pa},Z}^2 - P_{\text{pa},X}$ (5) (named W_C in the source)	$\mathbb{F}_p$
R_a	Mixed-addition scratch $R_a = T_Y \cdot P_{\text{pa},Z}^3 - P_{\text{pa},Y}$ (6) (named W_D in the source)	$\mathbb{F}_p$

Table 2: Witness columns (8 total). The single chained triple (P_X, P_Y, P_Z) holds both the row's input ($P[t]$) and the next row's input ($P[t+1] = P_X^{\downarrow}[t]$, etc.); there are no separate input/output columns.

per-constraint algebraic degree for fewer committed columns; the maximum constraint degree in the trace MLEs is 6.

Remark 3.3 (Conditional add via output-selection rather than gating). The per-row addition output is not committed; the conditional add P_{add}-or-P_{pa} is folded into the chaining via (10)–(12). This avoids the dedicated $T = (T_X : T_Y : T_Z)$ Jacobian-coordinate columns that an explicit *compute-then-gate* approach would require.

3.3 Constraints

There are 11 constraints in total: 3 doubling constraints, 2 mixed-addition scratch constraints, 3 output-selection constraints (which fold in the conditional add and the next-row chaining), and 3 init-boundary constraints. All constraints carry the zero ideal, i.e. they are strict equalities over $\mathbb{F}_p$.

Doubling constraints (anchored at every active row t). For every $t \in [0, 256)$ (gated by S_ACTIVE):

$$\mathsf{S_ACTIVE}[t] \cdot (P_{\text{pa},Z}[t] \;-\; 2\,P_Y[t]\,P_Z[t]) = 0, \tag{D2}$$

$$\mathsf{S_ACTIVE}[t] \cdot (P_{\text{pa},X}[t] \;-\; 9\,P_X[t]^4 \;+\; 8\,P_X[t]\,P_Y[t]^2) = 0, \tag{D3}$$

$$\mathsf{S_ACTIVE}[t] \cdot (P_{\text{pa},Y}[t] \;-\; 12\,P_X[t]^3\,P_Y[t]^2 \;+\; 3\,P_X[t]^2\,P_{\text{pa},X}[t] \;+\; 8\,P_Y[t]^4) = 0. \tag{D4}$$

Algebraic degrees in the trace MLEs (after gating): D2 deg 3, D3 deg 5, D4 deg 6 (via the $12P_X^3P_Y^2$ term).

Addition-scratch constraints. For every $t \in [0, 256)$:

$$\mathsf{S_ACTIVE}[t] \cdot (H[t] \; - \; \mathsf{PA_X_ADDEND}[t] \cdot P_{\mathrm{pa},Z}[t]^2 \; + \; P_{\mathrm{pa},X}[t]) = 0, \tag{A1}$$

$$\mathsf{S_ACTIVE}[t] \cdot (R_a[t] \; - \; \mathsf{PA_Y_ADDEND}[t] \cdot P_{\mathrm{pa},Z}[t]^3 \; + \; P_{\mathrm{pa},Y}[t]) = 0. \tag{A2}$$

Algebraic degrees (after gating): A1 deg 4, A2 deg 5.

Output-selection-and-chaining constraints. These three constraints inline the addition outputs of (7)–(9) into the conditional-add chaining (10)–(12). For every $t \in [0, 256)$:

$$\mathsf{S_ACTIVE}[t] \cdot \Big(P_X^{\downarrow}[t] - P_{\mathrm{pa},X}[t] - \mathsf{S_ADD}[t](R_a[t]^2 - H[t]^3 - 2P_{\mathrm{pa},X}[t]H[t]^2 - P_{\mathrm{pa},X}[t])\Big) = 0, \tag{O1}$$

$$\begin{aligned}\mathsf{S_ACTIVE}[t] \cdot \Big(P_Y^{\downarrow}[t] - P_{\mathrm{pa},Y}[t] - \mathsf{S_ADD}[t](3R_a[t]P_{\mathrm{pa},X}[t]H[t]^2 + R_a[t]H[t]^3 \\ - R_a[t]^3 - P_{\mathrm{pa},Y}[t]H[t]^3 - P_{\mathrm{pa},Y}[t])\Big) = 0,\end{aligned} \tag{O2}$$

$$\mathsf{S_ACTIVE}[t] \cdot \Big(P_Z^{\downarrow}[t] - P_{\mathrm{pa},Z}[t] - \mathsf{S_ADD}[t](P_{\mathrm{pa},Z}[t]H[t] - P_{\mathrm{pa},Z}[t])\Big) = 0. \tag{O3}$$

Algebraic degrees (after gating): O1 deg 5, O2 deg 6, O3 deg 4. The deg-6 monomial in O2 is exactly $\mathsf{S_ACTIVE} \cdot \mathsf{S_ADD} \cdot R_a \cdot P_{\mathrm{pa},X} \cdot H^2$ (Theorem 2.2).

Init-boundary constraints (anchored at row 0). The Jacobian state at row 0 is pinned to the verifier-supplied seed R_{init}:

$$\mathsf{S_INIT}[t] \cdot (P_X[t] - \mathsf{PA_R_INIT_X}[t]) = 0, \tag{I1}$$

$$\mathsf{S_INIT}[t] \cdot (P_Y[t] - \mathsf{PA_R_INIT_Y}[t]) = 0, \tag{I2}$$

$$\mathsf{S_INIT}[t] \cdot (P_Z[t] - \mathsf{PA_R_INIT_Z}[t]) = 0. \tag{I3}$$

Algebraic degrees (after gating): all three are deg 2.

No final-row constraints. There are no final-row constraints in this UAIR. As discussed in Section 2.5, the off-protocol affine readout ($R_x \neq \mathcal{O}$ and $R_x \equiv r \pmod{n}$) is performed by the verifier directly on the witness cells $(P_X[256], P_Y[256], P_Z[256])$, without any in-circuit enforcement.

3.4 Algebraic-degree summary

Constraint	Maximum degree (after gating)	Reason
D2	3	$\mathsf{S_ACTIVE} \cdot P_Y \cdot P_Z$
D3	5	$\mathsf{S_ACTIVE} \cdot P_X^4$
D4	6	$\mathsf{S_ACTIVE} \cdot P_X^3 \cdot P_Y^2$
A1	4	$\mathsf{S_ACTIVE} \cdot \mathsf{PA_X_ADDEND} \cdot P_{\mathrm{pa},Z}^2$
A2	5	$\mathsf{S_ACTIVE} \cdot \mathsf{PA_Y_ADDEND} \cdot P_{\mathrm{pa},Z}^3$
O1	5	$\mathsf{S_ACTIVE} \cdot \mathsf{S_ADD} \cdot P_{\mathrm{pa},X} \cdot H^2$
O2	6	$\mathsf{S_ACTIVE} \cdot \mathsf{S_ADD} \cdot R_a \cdot P_{\mathrm{pa},X} \cdot H^2$
O3	4	$\mathsf{S_ACTIVE} \cdot \mathsf{S_ADD} \cdot P_{\mathrm{pa},Z} \cdot H$
I1–I3	2	$\mathsf{S_INIT} \cdot P_{X/Y/Z}$
Global maximum	6	attained by D4 and O2

4 Arithmetization summary

We summarize the dimensions and constraint counts of the ECDSA UAIR$^+$ above.

Quantity	**Value**
Number of Shamir rounds	NUM_SHAMIR_ROUNDS = 256
Final row index	FINAL_ROW = 256
Active rows	257
Trace length $n = 2^\nu$	$2^9 = 512$ (smallest power of $2 \geq 257$)
Prime tuple $\mathbf{q}$	(p) (single $\mathbb{F}_p$ branch; $p =$ secp256k1 base prime)
Witness columns	**8** (all native $\mathbb{F}_p$, no range checks)
Public-input columns	**9**
Total trace columns	17
Lookup constraints	0
Affine-combination lookups	0
Constraint count	**11** (3 doubling, 2 add-scratch, 3 output-selection, 3 init-boundary)
Maximum constraint degree	**6** (after S_ACTIVE gating; Section 3.4)

4.1 Why 8 committed witness columns suffice

A more direct arithmetization of the spec's Jacobian double-and-add loop would commit, per row, separate columns for the doubled point P_{pa}, the addition output P_{add}, the squared intermediate $S = P_Y^2$, and the inversion-witness Z_{inv} for the final affine readout — totalling roughly 11–13 committed columns. The deployed UAIR shrinks this to 8 via three deliberate moves:

- **Inlined intermediates.** $S = P_Y^2$, P_Y^4, $P_{\text{pa},Z}^2$, $P_{\text{pa},Z}^3$, H^2, H^3, and $P_{\text{pa},X}H^2$ are all expanded inside the constraint expressions rather than committed (Theorem 3.2). Cost: max constraint degree rises from 5 to 6, but the global max is anyway 6 (from D4), so the bound is unchanged.

- **Conditional add via output-selection.** The addition output P_{add} is not committed; it is inlined into the three output-selection constraints (10)–(12), which fold the conditional add and the next-row chaining into one operation (Theorem 3.3). This removes three committed columns that an explicit *compute* P_{add}*, then gate* approach would need.

- **Off-protocol affine readout.** The final-row Jacobian-to-affine conversion is performed by the verifier off-protocol (Section 2.5). This removes the Z_{inv} column and the affine output cells (R_x, R_y), saving up to three committed columns, and also removes the non-infinity check $P_Z[256] \cdot Z_{\text{inv}}[256] = 1$ and the x-binding $P_X[256] \cdot Z_{\text{inv}}[256]^2 = R_x$ from the constraint set.

The retained witness columns $(P_X, P_Y, P_Z, P_{\text{pa},X}, P_{\text{pa},Y}, P_{\text{pa},Z}, H, R_a)$ are precisely those whose multiplicities in the constraint expressions make inlining infeasible (each appears in at least two constraint families).

4.2 What the public input absorbs

Two further design moves push work out of the witness and into the verifier's public-input construction. Both are noted earlier but worth collecting:

- u_1, u_2 **and their bit-decompositions** b_1, b_2 **absorbed as public input.** The verifier computes $u_1 = e \cdot s^{-1} \bmod n$ and $u_2 = r \cdot s^{-1} \bmod n$ off-protocol from the signature (r, s) and message e, then publishes b_1, b_2 along with off-protocol checks $\sum_t b_i[t] \cdot 2^{256-t} = u_i$ in $\mathbb{F}_n$ (Theorem 1.1). This eliminates the entire $\mathbb{F}_n$-trace and the bit-decomposition accumulator constraints; the loop reads the bits directly via S_ADD and the per-row addend.

- **Per-row addend coordinates absorbed as public input.** The verifier writes $T[t] = T_{b_1[t]\,b_2[t]} \in \{\mathcal{O}, G, Q, G+Q\}$ into the columns $(\mathsf{PA_X_ADDEND}, \mathsf{PA_Y_ADDEND})$ for each row, with $\mathsf{S_ADD}[t] = \mathbf{1}\{T[t] \neq \mathcal{O}\}$ (Theorem 2.1). This eliminates the four-way selector constraints that an in-circuit *compute the addend from the bits* approach would need, and keeps the addition-formula constraints at their stated degrees rather than pushing them up by the addend selector.

4.3 Composition with downstream UAIRs

The retained $\mathsf{S_FINAL}$ selector (which is otherwise unused in this UAIR) lets a downstream UAIR compose with the ECDSA UAIR by adding final-row constraints — for example, an affine-conversion UAIR that commits to $Z_{\text{inv}}[256]$ and enforces $P_Z[256] \cdot Z_{\text{inv}}[256] = 1$ in-circuit, or a gluing UAIR that binds the digest of an upstream SHA-256 UAIR to the ECDSA message e used to derive u_1. The implementation's `EcdsaUair` + `Sha256CompressionSliceUair` side-by-side merge follows exactly this pattern: the digest column $\mathsf{PA_E}$ of the SHA half is read out-of-band by the verifier, u_1 is computed off-protocol, and the resulting per-row $(\mathsf{PA_X_ADDEND}, \mathsf{PA_Y_ADDEND})$ are written into the ECDSA half's public input. The cross-binding is therefore *implicit*, enforced by the verifier supplying coherent public inputs rather than by an in-circuit constraint, which is the same composition pattern used by the SHA UAIR for its $\mathsf{PA_A}/\mathsf{PA_E}$ digest exposure.